CETAM

Centro de Educação Tecnológica do Amazonas

G A B A R I T O O F I C I A L

Linguagem de Programação

Python

Respostas das atividades da apostila do aluno

Projeto integrador: Agência Amazônia Tur Digital

EXCLUSIVO DO PROFESSOR

Curso de Qualificação Profissional • 80 horas • PPC versão 001/DAC-2026

Elaboração: Profa. Priscila Gonçalves

Manaus – Amazonas • 2026

Sumário

Orientações para a correção 2
Como usar 2
Convenções 2
Fase 1 – Descoberta e Fundamentos 3
Aula 1 – Abertura e conceituação 3
Aula 2 – Tipos de dados, variáveis, entrada e saída 4
Aula 3 – Operadores aritméticos 4
Aula 4 – Operadores relacionais e lógicos; estrutura de um algoritmo 6
Fase 2 – Decisão 9
Aula 5 – Estrutura condicional if / else 9
Aula 6 – elif, condições encadeadas e compostas 11
Aula 7 – Consolidação da Fase 2 e AV1 13
Fase 3 – Repetição 15
Aula 8 – Laço while 15
Aula 9 – Laço for e range() 16
Aula 10 – break, continue, pass, menu interativo e AV2 18
Fase 4 – Organização de Dados 21
Aula 11 – Listas 21
Aula 12 – Tuplas e dicionários 22
Aula 13 – Manipulação de strings 23
Aula 14 – Consolidação da Fase 4 e AV3 24
Fase 5 – Modularização e Persistência 27
Aula 15 – Funções com def 27
Aula 16 – Parâmetros, argumentos, retorno e escopo 27
Aula 17 – Arquivos .txt 31
Fase 6 – POO e Integração 34
Aula 18 – Fundamentos de Programação Orientada a Objetos 34
Aula 19 – Desenvolvimento do cofre digital (oficina de projeto) 35
Aula 20 – Apresentação final, AV4 e encerramento 38
Resumo das saídas-chave 40

Orientações para a correção

Este documento reúne as respostas de **todas as atividades** da apostila do aluno, organizadas por fase e por aula, além das **soluções de referência dos desafios do projeto** e das **matrizes de correção** das avaliações AV1 a AV4. Os códigos foram **executados com entradas de teste** e as saídas apresentadas são as obtidas nessa execução.

Como usar

- **Há mais de uma solução correta.** Compare o **comportamento** do programa do aluno com a saída esperada, e não a forma exata de escrever.
- **Teste com valores variados**, inclusive os limites (por exemplo, nota 7, total de R$ 1.000,00) e entradas inválidas.
- **Observe os erros recorrentes** (conversão de tipos, `=` no lugar de `==`, indentação, `range` sem o fim incluído) e use-os na devolutiva coletiva.
- Nas atividades de **explicação**, aceite respostas com as palavras do aluno desde que o conceito esteja correto.
- As **matrizes de correção** descrevem desempenhos esperados. A **atribuição de notas é de competência exclusiva do professor** (PPC, item 11).

Convenções

Elemento	Significado
Bloco cinza com borda verde	Código de resposta (solução de referência).
Bloco azul com borda azul	Saída esperada na execução de teste.
Quadro “Nota ao professor”	Dicas de correção e erros frequentes.
Valores em reais e destinos	Fictícios, criados para fins didáticos.

> ⚠ **Atenção**
> No **Colab**, os arquivos criados (`.txt`) ficam no ambiente temporário e desaparecem ao encerrar a sessão; no **VS Code**, eles são gravados na pasta do projeto aberta. Lembre os alunos disso nas atividades da Aula 17 em diante.

Fase 1 – Descoberta e Fundamentos

Aula 1 – Abertura e conceituação

Atividade 1

> *Escreva o pseudocódigo para "calcular o custo de um passeio de barco para um grupo".*

Resposta

```
INÍCIO
    Leia preço_por_pessoa
    Leia número_de_pessoas
    custo_total ← preço_por_pessoa × número_de_pessoas
    Mostre custo_total
FIM
```

> **Nota ao professor**
> Aceite qualquer redação em português, desde que apresente os três momentos: **entrada** (leitura dos dois dados), **processamento** (multiplicação) e **saída** (exibição do resultado), na ordem correta.

Atividade 2

> *No Colab, execute `print("Olá, Amazonas!")`. Depois, altere a mensagem para incluir seu nome.*

Resposta

```
print("Olá, Amazonas!")
print("Olá, Amazonas! Eu sou a Marina.")
```

Saída esperada:

```
Olá, Amazonas!
Olá, Amazonas! Eu sou a Marina.
```

> **Nota ao professor**
> A saída varia conforme o nome do aluno. Verifique se a célula foi executada (▶ ou Shift + Enter) e se o texto está entre aspas.

Atividade 3

> *No VS Code, crie o arquivo `ola.py` e execute-o pelo botão ▶ e pelo terminal.*

Resposta

Conteúdo de `ola.py`, executado pelo botão ▶ (Run Python File) e pelo terminal com o comando `python ola.py` (ou `py ola.py` no Windows, `python3 ola.py` no Linux/macOS):

```
print("Olá, Amazonas! Eu sou a Marina.")
```

Saída esperada:

```
Olá, Amazonas! Eu sou a Marina.
```

> **Nota ao professor**
> Erros frequentes: salvar o arquivo sem a extensão `.py`, não instalar a extensão Python da Microsoft e não marcar “Add python.exe to PATH” na instalação.

Aula 2 – Tipos de dados, variáveis, entrada e saída

Atividade 1

> *Crie um programa que receba nome, destino e dias de viagem e exiba a frase “Fulano viajará para X por Y dias.”*

Resposta

```
nome = input("Nome do cliente: ")
destino = input("Destino: ")
dias = int(input("Dias de viagem: "))
print(f"{nome} viajará para {destino} por {dias} dias.")
```

Saída esperada:

```
Nome do cliente: Marina
Destino: Presidente Figueiredo
Dias de viagem: 3
Marina viajará para Presidente Figueiredo por 3 dias.
```

> **Nota ao professor**
> Verifique a conversão de `dias` com `int()`. Soluções com concatenação (`nome + " viajará..."`) também são válidas, desde que `dias` seja convertido com `str()`.

Atividade 2

> *Caça aos erros: (a) `dias = input("Dias: ") + 1` (b) `1destino = "Parintins"` (c) `print("Total: " + 150)`.*

Resposta

(a) `input()` devolve texto (`str`) e não é possível somar texto com número (**TypeError**). Correção: `dias = int(input("Dias: ")) + 1`.

(b) Nomes de variáveis não podem começar com número (**SyntaxError**). Correção: `destino1 = "Parintins"` (ou `destino_1`).

(c) Não se concatena texto com número (**TypeError**). Correções possíveis: `print("Total: " + str(150))` ou `print(f"Total: {150}")`.

Aula 3 – Operadores aritméticos

Atividade 1

> *Leia três notas e mostre a média.*

Resposta

```
n1 = float(input("Nota 1: "))
n2 = float(input("Nota 2: "))
n3 = float(input("Nota 3: "))
media = (n1 + n2 + n3) / 3
print(f"Média: {media:.2f}")
```

Saída esperada:

```
Nota 1: 8
Nota 2: 7
Nota 3: 9
Média: 8.00
```

> **Nota ao professor**
> Erro típico: escrever `n1 + n2 + n3 / 3` sem parênteses (a divisão é feita antes da soma).

Atividade 2

> *Leia o valor de uma compra e o valor pago e calcule o troco.*

Resposta

```
compra = float(input("Valor da compra (R$): "))
pago = float(input("Valor pago (R$): "))
troco = pago - compra
print(f"Troco: R$ {troco:.2f}")
```

Saída esperada:

```
Valor da compra (R$): 87.5
Valor pago (R$): 100
Troco: R$ 12.50
```

Atividade 3

> *Com orçamento de R$ 1.000,00 e diária de R$ 330,00, mostre quantos dias inteiros cabem no orçamento e quanto sobra.*

Resposta

```
orcamento = 1000
diaria = 330
dias_inteiros = orcamento // diaria
sobra = orcamento % diaria
print(f"Dias inteiros: {dias_inteiros}")
print(f"Sobra: R$ {sobra:.2f}")
```

Saída esperada:

```
Dias inteiros: 3
Sobra: R$ 10.00
```

Atividade 4

> *Sem executar, calcule `2 + 3 * 4 ** 2`.*

Resposta

50. Primeiro a potência (`4 ** 2 = 16`), depois a multiplicação (`3 * 16 = 48`) e por fim a soma (`2 + 48 = 50`).

Desafio do projeto – Custo total da viagem

> *Peça dias, hospedagem, transporte e alimentação por dia; mostre o gasto diário e o total.*

Resposta

```
dias = int(input("Quantidade de dias: "))
hospedagem = float(input("Hospedagem por dia (R$): "))
transporte = float(input("Transporte por dia (R$): "))
alimentacao = float(input("Alimentação por dia (R$): "))

gasto_dia = hospedagem + transporte + alimentacao
total = dias * gasto_dia

print(f"Gasto por dia: R$ {gasto_dia:.2f}")
print(f"Total da viagem: R$ {total:.2f}")
```

Saída esperada:

```
Quantidade de dias: 3
Hospedagem por dia (R$): 200
Transporte por dia (R$): 50
Alimentação por dia (R$): 80
Gasto por dia: R$ 330.00
Total da viagem: R$ 990.00
```

Aula 4 – Operadores relacionais e lógicos; estrutura de um algoritmo

Atividade 1

> *Sendo `tem_orcamento = True` e `tem_vaga = False`, qual o resultado de `tem_orcamento and tem_vaga`, `tem_orcamento or tem_vaga` e `not tem_vaga`?*

Resposta

```
tem_orcamento = True
tem_vaga = False
print(tem_orcamento and tem_vaga)
print(tem_orcamento or tem_vaga)
print(not tem_vaga)
```

Saída esperada:

```
False
True
True
```

Nota ao professor

Resultado: `False`, `True` e `True`. O `and` exige as duas condições verdadeiras; o `or` basta uma.

Atividade 2

> *Escreva uma expressão que seja verdadeira quando `dias` estiver entre 1 e 30.*

Resposta

`1 <= dias <= 30`, ou, de forma equivalente, `dias >= 1 and dias <= 30`.

```
dias = int(input("Dias: "))
print(1 <= dias <= 30)
```

Saída esperada – dias = 15:

```
Dias: 15
True
```

Saída esperada – dias = 45:

```
Dias: 45
False
```

Desafio do projeto – Entrega da Fase 1 – Calculadora de viagem v1

> *Programa completo com entrada, processamento e saída.*

Resposta

```
# Agência Amazônia Tur Digital - Calculadora de viagem v1

# ----- ENTRADA -----
print("=== Agência Amazônia Tur Digital ===")
nome = input("Nome do cliente: ")
destino = input("Destino: ")
dias = int(input("Quantidade de dias: "))
hospedagem = float(input("Hospedagem por dia (R$): "))
transporte = float(input("Transporte por dia (R$): "))
alimentacao = float(input("Alimentação por dia (R$): "))

# ----- PROCESSAMENTO -----
gasto_dia = hospedagem + transporte + alimentacao
total = dias * gasto_dia

# ----- SAÍDA -----
print()
print(f"Cliente: {nome}")
print(f"Destino: {destino}")
print(f"Gasto por dia: R$ {gasto_dia:.2f}")
print(f"Total da viagem ({dias} dias): R$ {total:.2f}")
```

Saída esperada:

```
=== Agência Amazônia Tur Digital ===
Nome do cliente: Marina
Destino: Parintins
Quantidade de dias: 3
Hospedagem por dia (R$): 200
Transporte por dia (R$): 50
Alimentação por dia (R$): 80

Cliente: Marina
Destino: Parintins
```

```
Gasto por dia: R$ 330.00
Total da viagem (3 dias): R$ 990.00
```

Nota ao professor

Na revisão cruzada, observe: o programa roda? Os nomes são claros? Há comentários? Os tipos foram convertidos (`int` e `float`)?

Fase 2 – Decisão

Aula 5 – Estrutura condicional if / else

Atividade 1

> *Leia a idade e informe se a pessoa é maior de idade.*

Resposta

```
idade = int(input("Idade: "))
if idade >= 18:
    print("Maior de idade.")
else:
    print("Menor de idade.")
```

Saída esperada – idade = 20:

```
Idade: 20
Maior de idade.
```

Saída esperada – idade = 15:

```
Idade: 15
Menor de idade.
```

Atividade 2

> *Leia a nota e informe "Aprovado" (nota ≥ 7) ou "Reprovado".*

Resposta

```
nota = float(input("Nota: "))
if nota >= 7:
    print("Aprovado")
else:
    print("Reprovado")
```

Saída esperada – nota = 7.5:

```
Nota: 7.5
Aprovado
```

Saída esperada – nota = 5:

```
Nota: 5
Reprovado
```

> **Nota ao professor**
> Teste o valor limite (7): deve resultar em "Aprovado". Erro comum: usar `>` em vez de `>=`.

Atividade 3

> *Aplique 20% de desconto (fictício) a clientes com 60 anos ou mais ou estudantes.*

Resposta

```
idade = int(input("Idade: "))
estudante = input("É estudante? (s/n): ").lower()
```

```
valor = 150.00

if idade >= 60 or estudante == "s":
    valor = valor * 0.8
    print("Desconto de 20% aplicado.")
else:
    print("Valor integral.")

print(f"Valor do passeio: R$ {valor:.2f}")
```

Saída esperada – idoso:

```
Idade: 65
É estudante? (s/n): n
Desconto de 20% aplicado.
Valor do passeio: R$ 120.00
```

Saída esperada – estudante:

```
Idade: 30
É estudante? (s/n): s
Desconto de 20% aplicado.
Valor do passeio: R$ 120.00
```

Saída esperada – sem desconto:

```
Idade: 30
É estudante? (s/n): n
Valor integral.
Valor do passeio: R$ 150.00
```

> **Nota ao professor**
> Observe o uso de `or` (basta uma das condições). Se o aluno usar `and`, só terá desconto quem for idoso **e** estudante.

Desafio do projeto – Dentro ou fora do orçamento?

> *Informe se o total está dentro ou fora do orçamento de R$ 1.000,00 e, se estiver fora, quanto excede.*

Resposta

```
LIMITE = 1000.00

dias = int(input("Quantidade de dias: "))
gasto_dia = float(input("Gasto por dia (R$): "))
total = dias * gasto_dia
print(f"Total da viagem: R$ {total:.2f}")

if total <= LIMITE:
    print("Viagem dentro do orçamento de R$ 1000,00.")
else:
    excesso = total - LIMITE
    print(f"Viagem fora do orçamento! Excede em R$ {excesso:.2f}.")
```

Saída esperada – dentro:

```
Quantidade de dias: 3
Gasto por dia (R$): 330
Total da viagem: R$ 990.00
Viagem dentro do orçamento de R$ 1000,00.
```

Saída esperada – fora:

```
Quantidade de dias: 5
Gasto por dia (R$): 400
Total da viagem: R$ 2000.00
Viagem fora do orçamento! Excede em R$ 1000.00.
```

Aula 6 – elif, condições encadeadas e compostas

Atividade 1

> *Classifique a média de um aluno: ≥ 7 "Aprovado", ≥ 5 "Recuperação", senão "Reprovado".*

Resposta

```
media = float(input("Média: "))
if media >= 7:
    print("Aprovado")
elif media >= 5:
    print("Recuperação")
else:
    print("Reprovado")
```

Saída esperada – média 8:

```
Média: 8
Aprovado
```

Saída esperada – média 6:

```
Média: 6
Recuperação
```

Saída esperada – média 3:

```
Média: 3
Reprovado
```

Atividade 2

> *Um cliente ganha cortesia (um passeio grátis) se `dias >= 5` e `total <= 1500`. Escreva a condição.*

Resposta

A condição é `dias >= 5 and total <= 1500`. Exemplo completo:

```
dias = int(input("Dias: "))
total = float(input("Total (R$): "))

if dias >= 5 and total <= 1500:
    print("Cliente ganha cortesia: um passeio grátis!")
else:
    print("Sem cortesia.")
```

Saída esperada – ganha cortesia:

```
Dias: 6
Total (R$): 1200
Cliente ganha cortesia: um passeio grátis!
```

Saída esperada – total alto:

```
Dias: 6
Total (R$): 1800
Sem cortesia.
```

Saída esperada – poucos dias:

```
Dias: 3
Total (R$): 900
Sem cortesia.
```

Desafio do projeto – Classificação por faixa e sugestão de destino

> *Classifique a viagem (Econômico, Conforto ou Premium) e sugira um destino compatível.*

Resposta

```
dias = int(input("Dias: "))
gasto_dia = float(input("Gasto por dia (R$): "))
total = dias * gasto_dia

if total <= 800:
    pacote = "Econômico"
elif total <= 1500:
    pacote = "Conforto"
else:
    pacote = "Premium"

if pacote == "Econômico" and dias <= 3:
    sugestao = "Presidente Figueiredo"
elif pacote == "Conforto":
    sugestao = "Novo Airão"
else:
    sugestao = "Parintins"

print(f"Total: R$ {total:.2f} - Pacote {pacote}")
print(f"Sugestão de destino: {sugestao}")
```

Saída esperada – econômico:

```
Dias: 2
Gasto por dia (R$): 300
Total: R$ 600.00 - Pacote Econômico
Sugestão de destino: Presidente Figueiredo
```

Saída esperada – conforto:

```
Dias: 4
Gasto por dia (R$): 300
Total: R$ 1200.00 - Pacote Conforto
Sugestão de destino: Novo Airão
```

Saída esperada – premium:

```
Dias: 6
```

```
Gasto por dia (R$): 400
Total: R$ 2400.00 - Pacote Premium
Sugestão de destino: Parintins
```

> **Nota ao professor**
> As faixas e os destinos sugeridos são livres: valorize a **coerência** entre faixa e sugestão. Aceite outras escolhas de destino.

Aula 7 – Consolidação da Fase 2 e AV1

AV1 – Modelo de resolução (Calculadora v2)

> *Desenvolver o programa da agência: entrada de nome, dias e gasto por dia, cálculo do total, decisão sobre o orçamento e `elif` para a faixa do pacote.*

Resposta

```
# Agência Amazônia Tur Digital - Calculadora de viagem v2
LIMITE = 1000.00

nome = input("Nome do cliente: ")
dias = int(input("Quantidade de dias: "))
gasto_dia = float(input("Gasto por dia (R$): "))

total = dias * gasto_dia

if total <= 800:
    pacote = "Econômico"
elif total <= 1500:
    pacote = "Conforto"
else:
    pacote = "Premium"

print(f"\nCliente: {nome}")
print(f"Total: R$ {total:.2f} - Pacote {pacote}")

if total <= LIMITE:
    print("Dentro do orçamento de R$ 1000,00.")
else:
    print(f"Fora do orçamento! Excede em R$ {total - LIMITE:.2f}.")
```

Saída esperada – Conforto, dentro do orçamento:

```
Nome do cliente: Marina
Quantidade de dias: 3
Gasto por dia (R$): 330

Cliente: Marina
Total: R$ 990.00 - Pacote Conforto
Dentro do orçamento de R$ 1000,00.
```

Saída esperada – Premium, fora do orçamento:

```
Nome do cliente: Carlos
```

```
Quantidade de dias: 5
Gasto por dia (R$): 400

Cliente: Carlos
Total: R$ 2000.00 - Pacote Premium
Fora do orçamento! Excede em R$ 1000.00.
```

Saída esperada – Econômico, dentro do orçamento:

```
Nome do cliente: Ana
Quantidade de dias: 2
Gasto por dia (R$): 300

Cliente: Ana
Total: R$ 600.00 - Pacote Econômico
Dentro do orçamento de R$ 1000,00.
```

AV1 – Matriz de correção

Resposta

Os descritores abaixo orientam a correção. A atribuição de notas é de competência exclusiva do professor.

Critério	Atende plenamente	Atende parcialmente	Não atende
Funcionamento	Roda sem erros e resultados corretos nos três cenários de teste.	Roda, mas erra em algum cenário (ex.: limite).	Não executa ou não produz o resultado.
Lógica	`if/elif/else` corretos, faixas e orçamento coerentes.	Faixas ou orçamento com falha parcial.	Decisões ausentes ou incorretas.
Legibilidade	Nomes claros, indentação correta e comentários.	Nomes pouco claros ou sem comentários.	Código confuso, sem organização.
Tratamento dos tipos	`int()` e `float()` aplicados corretamente.	Conversão em parte das entradas.	Sem conversões; erros de tipo.

Fase 3 – Repetição

Aula 8 – Laço while

Atividade 1

> *Peça uma nota entre 0 e 10 e repita a pergunta até o valor ser válido.*

Resposta

```
nota = float(input("Nota (0 a 10): "))
while nota < 0 or nota > 10:
    print("Nota inválida! Digite um valor entre 0 e 10.")
    nota = float(input("Nota (0 a 10): "))
print(f"Nota registrada: {nota}")
```

Saída esperada:

```
Nota (0 a 10): 11
Nota inválida! Digite um valor entre 0 e 10.
Nota (0 a 10): -2
Nota inválida! Digite um valor entre 0 e 10.
Nota (0 a 10): 8.5
Nota registrada: 8.5
```

> **Nota ao professor**
> O operador deve ser `or` na condição de invalidez. Com `and`, a condição nunca seria verdadeira.

Atividade 2

> *Some gastos digitados pelo usuário até que ele digite 0.*

Resposta

```
soma = 0
gasto = float(input("Gasto (0 para encerrar): "))
while gasto != 0:
    soma += gasto
    gasto = float(input("Gasto (0 para encerrar): "))
print(f"Total dos gastos: R$ {soma:.2f}")
```

Saída esperada:

```
Gasto (0 para encerrar): 330
Gasto (0 para encerrar): 200
Gasto (0 para encerrar): 150.5
Gasto (0 para encerrar): 0
Total dos gastos: R$ 680.50
```

Desafio do projeto – Várias viagens em sequência

> *Repetir o cálculo, perguntando "Deseja continuar? (s/n)", com validação dos dias (1 a 30).*

Resposta

```
continuar = "s"
while continuar == "s":
```

```
    dias = int(input("Dias de viagem (1 a 30): "))
    while dias < 1 or dias > 30:
        print("Valor inválido! Digite de 1 a 30.")
        dias = int(input("Dias de viagem (1 a 30): "))
    gasto_dia = float(input("Gasto por dia (R$): "))
    print(f"Total: R$ {dias * gasto_dia:.2f}")
    continuar = input("Deseja continuar? (s/n): ").lower()
print("Obrigado por usar a calculadora!")
```

Saída esperada:

```
Dias de viagem (1 a 30): 40
Valor inválido! Digite de 1 a 30.
Dias de viagem (1 a 30): 3
Gasto por dia (R$): 330
Total: R$ 990.00
Deseja continuar? (s/n): s
Dias de viagem (1 a 30): 2
Gasto por dia (R$): 200
Total: R$ 400.00
Deseja continuar? (s/n): n
Obrigado por usar a calculadora!
```

Aula 9 – Laço for e range()

Atividade 1

> *Mostre a tabuada de um número lido (de 1 a 10).*

Resposta

```
n = int(input("Tabuada de qual número? "))
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

Saída esperada:

```
Tabuada de qual número? 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Nota ao professor

Erro comum: `range(1, 10)` (para no 9). O fim do `range` não é incluído.

Atividade 2

> *Some as diárias de 1 a `dias`, sendo que a diária do dia 1 é R$ 100 e aumenta R$ 20 por dia.*

Resposta

```
dias = int(input("Dias: "))
diaria = 100
soma = 0

for dia in range(1, dias + 1):
    print(f"Dia {dia}: R$ {diaria:.2f}")
    soma += diaria
    diaria += 20

print(f"Soma das diárias: R$ {soma:.2f}")
```

Saída esperada:

```
Dias: 3
Dia 1: R$ 100.00
Dia 2: R$ 120.00
Dia 3: R$ 140.00
Soma das diárias: R$ 360.00
```

Atividade 3

> *Faça uma contagem regressiva de 10 até 1 e, no final, mostre "Embarcar!".*

Resposta

```
for i in range(10, 0, -1):
    print(i)
print("Embarcar!")
```

Saída esperada:

```
10
9
8
7
6
5
4
3
2
1
Embarcar!
```

Desafio do projeto – Extrato da viagem

> *Exibir o gasto de cada dia e o valor acumulado.*

Resposta

```
dias = int(input("Dias: "))
gasto_dia = float(input("Gasto por dia (R$): "))
acumulado = 0

for dia in range(1, dias + 1):
    acumulado += gasto_dia
    print(f"Dia {dia}: R$ {gasto_dia:.2f} | Acumulado: R$ {acumulado:.2f}")
```

Saída esperada:

```
Dias: 3
Gasto por dia (R$): 330
Dia 1: R$ 330.00 | Acumulado: R$ 330.00
Dia 2: R$ 330.00 | Acumulado: R$ 660.00
Dia 3: R$ 330.00 | Acumulado: R$ 990.00
```

Aula 10 – break, continue, pass, menu interativo e AV2

Atividade 1

> *Em um `for` de 1 a 5, se o número for 3 use `continue`; se for 5 use `break`; nos demais casos, mostre o número. Qual é a saída?*

Resposta

```
for numero in range(1, 6):
    if numero == 3:
        continue
    if numero == 5:
        break
    print(numero)
```

Saída esperada:

```
1
2
4
```

> **Nota ao professor**
> Saída: 1, 2 e 4. O 3 é pulado (`continue`) e o laço termina no 5 antes de imprimi-lo (`break`).

Atividade 2

> *Acrescente ao menu a opção "3 – Ver orçamento limite" que mostre o valor de R$ 1.000,00.*

Resposta

```
LIMITE = 1000.00
destinos = ["Parintins", "Novo Airão", "Presidente Figueiredo", "Maués"]

while True:
    print("\n=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===")
    print("1 - Calcular viagem")
    print("2 - Ver destinos")
    print("3 - Ver orçamento limite")
    print("0 - Sair")
    opcao = input("Escolha uma opção: ")

    if opcao == "1":
        dias = int(input("Dias: "))
        gasto_dia = float(input("Gasto por dia: "))
        print(f"Total: R$ {dias * gasto_dia:.2f}")
    elif opcao == "2":
        for destino in destinos:
```

```
            print("-", destino)
    elif opcao == "3":
        print(f"Orçamento limite: R$ {LIMITE:.2f}")
    elif opcao == "0":
        print("Até logo!")
        break
    else:
        print("Opção inválida!")
        continue
```

Saída esperada:

```
=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===
1 - Calcular viagem
2 - Ver destinos
3 - Ver orçamento limite
0 - Sair
Escolha uma opção: 3
Orçamento limite: R$ 1000.00

=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===
1 - Calcular viagem
2 - Ver destinos
3 - Ver orçamento limite
0 - Sair
Escolha uma opção: 1
Dias: 3
Gasto por dia: 330
Total: R$ 990.00

=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===
1 - Calcular viagem
2 - Ver destinos
3 - Ver orçamento limite
0 - Sair
Escolha uma opção: 2
- Parintins
- Novo Airão
- Presidente Figueiredo
- Maués

=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===
1 - Calcular viagem
2 - Ver destinos
3 - Ver orçamento limite
0 - Sair
Escolha uma opção: 9
Opção inválida!

=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===
1 - Calcular viagem
2 - Ver destinos
3 - Ver orçamento limite
0 - Sair
Escolha uma opção: 0
```

```
Até logo!
```

Nota ao professor
Este código também é o modelo de referência do **menu interativo** (entrega da Fase 3) e da **AV2**.

AV2 – Matriz de correção

Resposta

A AV2 é feita em duplas; avalie também o registro de colaboração de cada integrante.

Critério	Atende plenamente	Atende parcialmente	Não atende
Funcionamento	Menu repete até a opção de sair; todas as opções executam.	Menu funciona, mas alguma opção falha.	Menu não repete ou não executa.
Validação de entrada	Entradas inválidas tratadas sem encerrar o programa.	Validação apenas em parte das entradas.	Sem validação.
Controle de fluxo	`break` e `continue` usados no lugar certo.	Uso parcial ou desnecessário.	Ausente ou incorreto.
Colaboração	Tarefas divididas e contribuição de cada um registrada.	Registro incompleto ou desigual.	Sem registro de colaboração.

Fase 4 – Organização de Dados

Aula 11 – Listas

Atividade 1

> *Crie uma lista com 4 destinos. Mostre o primeiro e o último, adicione um quinto e ordene a lista.*

Resposta

```
destinos = ["Parintins", "Novo Airão", "Maués", "Presidente Figueiredo"]
print(destinos[0], destinos[-1])

destinos.append("Encontro das Águas")
destinos.sort()
print(destinos)
```

Saída esperada:

```
Parintins Presidente Figueiredo
['Encontro das Águas', 'Maués', 'Novo Airão', 'Parintins', 'Presidente
Figueiredo']
```

Atividade 2

> *Dada a lista de gastos `[990, 1250.5, 780]`, mostre o maior, o menor e a média.*

Resposta

```
gastos = [990, 1250.5, 780]
print(f"Maior: R$ {max(gastos):.2f}")
print(f"Menor: R$ {min(gastos):.2f}")
print(f"Média: R$ {sum(gastos) / len(gastos):.2f}")
```

Saída esperada:

```
Maior: R$ 1250.50
Menor: R$ 780.00
Média: R$ 1006.83
```

Desafio do projeto – Cadastro de destinos e média de gastos

> *Cadastrar vários destinos e gastos em duas listas e calcular a média.*

Resposta

```
destinos = []
gastos = []

continuar = "s"
while continuar == "s":
    destinos.append(input("Destino: "))
    gastos.append(float(input("Gasto total (R$): ")))
    continuar = input("Cadastrar outro? (s/n): ").lower()

print(f"Destinos cadastrados: {destinos}")
print(f"Média dos gastos: R$ {sum(gastos) / len(gastos):.2f}")
```

Saída esperada:

```
Destino: Parintins
Gasto total (R$): 990
Cadastrar outro? (s/n): s
Destino: Maués
Gasto total (R$): 1350
Cadastrar outro? (s/n): n
Destinos cadastrados: ['Parintins', 'Maués']
Média dos gastos: R$ 1170.00
```

Aula 12 – Tuplas e dicionários

Atividade 1

> *Crie um dicionário com nome, preço e duração de um passeio e percorra-o com `.items()`.*

Resposta

```
passeio = {"nome": "Passeio de barco", "preco": 120.00, "duracao": 3}

for chave, valor in passeio.items():
    print(f"{chave}: {valor}")
```

Saída esperada:

```
nome: Passeio de barco
preco: 120.0
duracao: 3
```

Atividade 2

> *Explique a diferença entre uma lista e uma tupla e dê um exemplo de uso de cada uma.*

Resposta

A **lista** (`[ ]`) é **mutável**: é possível adicionar, remover e alterar itens. Exemplo: a lista de clientes cadastrados, que muda a todo momento.

A **tupla** (`( )`) é **imutável**: depois de criada, não pode ser alterada. Exemplo: as coordenadas de uma cidade, `(-3.119, -60.021)`, que são dados fixos.

Desafio do projeto – Catálogo de pacotes da agência

> *Montar o catálogo em um dicionário e exibi-lo com `.items()`.*

Resposta

```
catalogo = {
    "Parintins": {"preco": 950.00, "dias": 3},
    "Novo Airão": {"preco": 1200.00, "dias": 4},
    "Presidente Figueiredo": {"preco": 780.00, "dias": 2},
    "Maués": {"preco": 1350.00, "dias": 4},
}

for nome, dados in catalogo.items():
    print(f"{nome}: R$ {dados['preco']:.2f} em {dados['dias']} dias")
```

Saída esperada:

```
Parintins: R$ 950.00 em 3 dias
Novo Airão: R$ 1200.00 em 4 dias
Presidente Figueiredo: R$ 780.00 em 2 dias
Maués: R$ 1350.00 em 4 dias
```

Aula 13 – Manipulação de strings

Atividade 1

> *Padronize o texto `"  joão PEDRO  "` para `"João Pedro"`.*

Resposta

```
texto = "  joão PEDRO  "
print(texto.strip().title())
```

Saída esperada:

```
João Pedro
```

> **Nota ao professor**
> `strip()` remove os espaços das pontas e `title()` coloca a inicial de cada palavra em maiúscula.

Atividade 2

> *Valide um e-mail digitado (deve conter `@` e `.`).*

Resposta

```
email = input("E-mail: ").strip().lower()
if "@" in email and "." in email:
    print("E-mail com formato aceitável.")
else:
    print("E-mail inválido.")
```

Saída esperada – e-mail válido:

```
E-mail: marina@email.com
E-mail com formato aceitável.
```

Saída esperada – sem @:

```
E-mail: marina.email.com
E-mail inválido.
```

Atividade 3

> *Gere o código de reserva no formato `PAR-03-MA` a partir do destino, dos dias e do nome do cliente.*

Resposta

```
nome = "marina costa"
destino = "Parintins"
dias = 3

codigo = f"{destino[:3].upper()}-{dias:02d}-{nome[:2].upper()}"
print(codigo)
```

Saída esperada:

```
PAR-03-MA
```

Desafio do projeto – Cadastro padronizado e recibo

> *Padronizar o nome do cliente e montar o recibo da viagem com f-strings.*

Resposta

```
nome = input("Nome do cliente: ").strip().title()
destino = input("Destino: ").strip().title()
dias = int(input("Dias: "))
total = float(input("Total (R$): "))

codigo = f"{destino[:3].upper()}-{dias:02d}-{nome[:2].upper()}"

print("=" * 34)
print("     RECIBO - AMAZÔNIA TUR DIGITAL")
print("=" * 34)
print(f"Cliente: {nome}")
print(f"Destino: {destino}")
print(f"Duração: {dias} dias")
print(f"Total:   R$ {total:.2f}")
print(f"Reserva: {codigo}")
```

Saída esperada:

```
Nome do cliente:   marina COSTA
Destino: parintins
Dias: 3
Total (R$): 990
==================================
     RECIBO - AMAZÔNIA TUR DIGITAL
==================================
Cliente: Marina Costa
Destino: Parintins
Duração: 3 dias
Total:   R$ 990.00
Reserva: PAR-03-MA
```

Aula 14 – Consolidação da Fase 4 e AV3

Atividade 1

> *Dada `nomes = ["Ana", "Bia", "Caio"]`, o que `nomes[-1]` e `nomes[0:2]` mostram?*

Resposta

```
nomes = ["Ana", "Bia", "Caio"]
print(nomes[-1])
print(nomes[0:2])
```

Saída esperada:

```
Caio
['Ana', 'Bia']
```

> **Nota ao professor**
> `[-1]` é o último item; o fatiamento `[0:2]` pega as posições 0 e 1 (o fim não é incluído).

Atividade 2

> *Qual a diferença entre `lista.append(x)` e `lista.remove(x)`?*

Resposta

`append(x)` **adiciona** `x` ao final da lista; `remove(x)` **retira** a primeira ocorrência de `x`. (Se `x` não existir na lista, `remove` gera `ValueError`.)

Atividade 3

> *Dado `aluno = {"nome": "Ana", "nota": 9}`, como alterar a nota para 10? E como acrescentar a chave `"curso"`?*

Resposta

```
aluno = {"nome": "Ana", "nota": 9}
aluno["nota"] = 10
aluno["curso"] = "Python"
print(aluno)
```

Saída esperada:

```
{'nome': 'Ana', 'nota': 10, 'curso': 'Python'}
```

Atividade 4

> *O que `"  Maués ".strip().upper()` devolve?*

Resposta

```
print("  Maués ".strip().upper())
```

Saída esperada:

```
MAUÉS
```

> **Nota ao professor**
> Resposta: `"MAUÉS"`. Os métodos são aplicados em sequência, da esquerda para a direita.

AV3 – Modelo de resolução do problema em laboratório

> *Cadastro de clientes com dicionário e lista.*

Resposta

```
clientes = []

continuar = "s"
while continuar == "s":
    cliente = {
        "nome": input("Nome: ").strip().title(),
        "destino": input("Destino: ").strip().title(),
        "dias": int(input("Dias: "))
```

```
    }
    clientes.append(cliente)
    continuar = input("Cadastrar outro? (s/n): ").lower()

print()
for cl in clientes:
    print(f"{cl['nome']} -> {cl['destino']} ({cl['dias']} dias)")
print(f"Total de clientes: {len(clientes)}")
```

Saída esperada:

```
Nome:  marina costa
Destino: parintins
Dias: 3
Cadastrar outro? (s/n): s
Nome: joão pedro
Destino: maués
Dias: 4
Cadastrar outro? (s/n): n

Marina Costa -> Parintins (3 dias)
João Pedro -> Maués (4 dias)
Total de clientes: 2
```

Nota ao professor

Questões objetivas e discursivas da AV3: elabore a partir dos conteúdos das aulas 11 a 13. Itens para correção do problema prático: lista de dicionários, uso de `append`, padronização com `strip`/`title` e percurso com `for`.

Fase 5 – Modularização e Persistência

Aula 15 – Funções com def

Atividade 1

> *Crie a função `saudacao_cliente(nome)` que mostre "Olá, [nome]! Boa viagem!".*

Resposta

```
def saudacao_cliente(nome):
    print(f"Olá, {nome}! Boa viagem!")

saudacao_cliente("Marina")
```

Saída esperada:

```
Olá, Marina! Boa viagem!
```

Atividade 2

> *Transforme o cálculo do total (dias × gasto por dia) em uma função e chame-a duas vezes com valores diferentes.*

Resposta

```
def calcular_total(dias, gasto_dia):
    return dias * gasto_dia

print(calcular_total(3, 330))
print(calcular_total(5, 200))
```

Saída esperada:

```
990
1000
```

> **Nota ao professor**
> Verifique o uso de `return` (e não só `print`) dentro da função.

Aula 16 – Parâmetros, argumentos, retorno e escopo

Atividade 1

> *Crie `eh_par(n)` que devolva `True` ou `False`.*

Resposta

```
def eh_par(n):
    return n % 2 == 0

print(eh_par(8))
print(eh_par(7))
```

Saída esperada:

```
True
False
```

Atividade 2

> *Crie `buscar_destino(nome, catalogo)` e teste com um nome existente e outro inexistente.*

Resposta

```
catalogo = {
    "Parintins": {"preco": 950.00, "dias": 3},
    "Maués": {"preco": 1350.00, "dias": 4},
}

def buscar_destino(nome, catalogo):
    return catalogo.get(nome.strip().title())

print(buscar_destino("maués", catalogo))
print(buscar_destino("Xingu", catalogo))
```

Saída esperada:

```
{'preco': 1350.0, 'dias': 4}
None
```

> **Nota ao professor**
> `dicionario.get(chave)` devolve `None` quando a chave não existe, evitando o `KeyError`.

Atividade 3

> *Explique com suas palavras a diferença entre `print` e `return`.*

Resposta

`print` apenas **mostra** o valor na tela; o programa não consegue usá-lo depois. `return` **devolve** o valor para quem chamou a função, que pode guardá-lo em uma variável, compará-lo ou usá-lo em outros cálculos. Uma função sem `return` devolve `None`.

Desafio do projeto – Sistema refatorado em funções (busca de destino e validação do orçamento)

> *Integrar `calcular_total`, `validar_orcamento`, `buscar_destino` e `cadastrar_cliente` ao menu.*

Resposta

```
CATALOGO = {
    "Parintins": {"preco": 950.00, "dias": 3},
    "Novo Airão": {"preco": 1200.00, "dias": 4},
    "Presidente Figueiredo": {"preco": 780.00, "dias": 2},
    "Maués": {"preco": 1350.00, "dias": 4},
}

def exibir_menu():
    print("\n=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===")
```

```
    print("1 - Calcular viagem")
    print("2 - Cadastrar cliente")
    print("3 - Listar clientes")
    print("4 - Buscar destino")
    print("0 - Sair")
    return input("Escolha: ")

def calcular_total(dias, gasto_dia):
    return dias * gasto_dia

def validar_orcamento(total, limite=1000.00):
    return total <= limite

def buscar_destino(nome):
    return CATALOGO.get(nome.strip().title())

def cadastrar_cliente(clientes):
    nome = input("Nome: ").strip().title()
    destino = input("Destino: ").strip().title()
    dias = int(input("Dias: "))
    clientes.append({"nome": nome, "destino": destino, "dias": dias})
    print("Cliente cadastrado!")

clientes = []
while True:
    opcao = exibir_menu()
    if opcao == "1":
        dias = int(input("Dias: "))
        gasto = float(input("Gasto por dia: "))
        total = calcular_total(dias, gasto)
        print(f"Total: R$ {total:.2f}")
        if validar_orcamento(total):
            print("Dentro do orçamento.")
        else:
            print("Fora do orçamento.")
    elif opcao == "2":
        cadastrar_cliente(clientes)
    elif opcao == "3":
        for cl in clientes:
            print(f"{cl['nome']} -> {cl['destino']} ({cl['dias']} dias)")
    elif opcao == "4":
        dados = buscar_destino(input("Destino: "))
        if dados is None:
            print("Destino não encontrado.")
        else:
            print(f"Preço: R$ {dados['preco']:.2f} | {dados['dias']} dias")
    elif opcao == "0":
        break
    else:
        print("Opção inválida!")
```

Saída esperada:

```
=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===
1 - Calcular viagem
```

```
2 - Cadastrar cliente
3 - Listar clientes
4 - Buscar destino
0 - Sair
Escolha: 1
Dias: 3
Gasto por dia: 330
Total: R$ 990.00
Dentro do orçamento.

=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===
1 - Calcular viagem
2 - Cadastrar cliente
3 - Listar clientes
4 - Buscar destino
0 - Sair
Escolha: 2
Nome: marina costa
Destino: parintins
Dias: 3
Cliente cadastrado!

=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===
1 - Calcular viagem
2 - Cadastrar cliente
3 - Listar clientes
4 - Buscar destino
0 - Sair
Escolha: 3
Marina Costa -> Parintins (3 dias)

=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===
1 - Calcular viagem
2 - Cadastrar cliente
3 - Listar clientes
4 - Buscar destino
0 - Sair
Escolha: 4
Destino: maués
Preço: R$ 1350.00 | 4 dias

=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===
1 - Calcular viagem
2 - Cadastrar cliente
3 - Listar clientes
4 - Buscar destino
0 - Sair
Escolha: 4
Destino: xingu
Destino não encontrado.

=== AGÊNCIA AMAZÔNIA TUR DIGITAL ===
1 - Calcular viagem
2 - Cadastrar cliente
```

```
3 - Listar clientes
4 - Buscar destino
0 - Sair
Escolha: 0
```

> **Nota ao professor**
> Use o checklist de code review da apostila: cada função faz uma só coisa? Há `return` onde deve devolver valor? Há código repetido?

Aula 17 – Arquivos .txt

Atividade 1

> *Grave uma lista de 5 destinos em `destinos.txt` e depois leia e mostre cada um.*

Resposta

```
destinos = ["Parintins", "Novo Airão", "Maués", "Presidente Figueiredo",
"Manaus"]

with open("destinos.txt", "w", encoding="utf-8") as arquivo:
    for destino in destinos:
        arquivo.write(destino + "\n")

with open("destinos.txt", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        print(linha.strip())
```

Saída esperada:

```
Parintins
Novo Airão
Maués
Presidente Figueiredo
Manaus
```

Atividade 2

> *Leia o arquivo `destinos.txt` e mostre quantas linhas ele possui.*

Resposta

```
with open("destinos.txt", "r", encoding="utf-8") as arquivo:
    linhas = arquivo.readlines()

print(f"O arquivo possui {len(linhas)} linhas.")
```

Saída esperada:

```
O arquivo possui 5 linhas.
```

> **Nota ao professor**
> Pressupõe que o arquivo da Atividade 1 já exista na mesma pasta. No VS Code, confirme que a **pasta** do projeto está aberta; no Colab, o arquivo some se a sessão for encerrada.

Atividade 3

> *Qual a diferença entre os modos `"w"` e `"a"`? O que acontece se usarmos `"w"` por engano em um arquivo com dados?*

Resposta

O modo `"w"` (escrita) cria o arquivo e **apaga todo o conteúdo anterior** se ele já existir; o modo `"a"` (acréscimo) mantém o conteúdo e **adiciona ao final**. Usar `"w"` por engano em um arquivo com dados **apaga esses dados**.

Desafio do projeto – Persistência de clientes e reservas

> *Carregar os dados ao iniciar e salvar a cada cadastro.*

Resposta

```
def salvar_clientes(clientes, arquivo="clientes.txt"):
    with open(arquivo, "w", encoding="utf-8") as f:
        for cl in clientes:
            f.write(f"{cl['nome']};{cl['destino']};{cl['dias']}\n")

def carregar_clientes(arquivo="clientes.txt"):
    clientes = []
    try:
        with open(arquivo, "r", encoding="utf-8") as f:
            for linha in f:
                nome, destino, dias = linha.strip().split(";")
                clientes.append({"nome": nome, "destino": destino, "dias":
int(dias)})
    except FileNotFoundError:
        pass
    return clientes

clientes = carregar_clientes()

while True:
    print("\n1 - Cadastrar cliente | 2 - Listar clientes | 0 - Sair")
    opcao = input("Escolha: ")
    if opcao == "1":
        nome = input("Nome: ").strip().title()
        destino = input("Destino: ").strip().title()
        dias = int(input("Dias: "))
        clientes.append({"nome": nome, "destino": destino, "dias": dias})
        salvar_clientes(clientes)
        print("Cliente cadastrado e salvo!")
    elif opcao == "2":
        for cl in clientes:
            print(f"{cl['nome']} -> {cl['destino']} ({cl['dias']} dias)")
    elif opcao == "0":
        break
    else:
        print("Opção inválida!")
```

Saída esperada – 1ª execução (cadastra e sai):

```
1 - Cadastrar cliente | 2 - Listar clientes | 0 - Sair
```

```
Escolha: 1
Nome: joão pedro
Destino: maués
Dias: 4
Cliente cadastrado e salvo!

1 - Cadastrar cliente | 2 - Listar clientes | 0 - Sair
Escolha: 0
```

Saída esperada – 2ª execução (os dados foram carregados do arquivo):

```
1 - Cadastrar cliente | 2 - Listar clientes | 0 - Sair
Escolha: 2
João Pedro -> Maués (4 dias)

1 - Cadastrar cliente | 2 - Listar clientes | 0 - Sair
Escolha: 0
```

Nota ao professor

A prova da persistência é fechar o programa e abri-lo de novo: os clientes devem aparecer. Se não aparecerem, verifique o modo de abertura, o nome do arquivo e a pasta de trabalho.

Fase 6 – POO e Integração

Aula 18 – Fundamentos de Programação Orientada a Objetos

Atividade 1

> *Crie a classe `Pacote` com os atributos `destino`, `preco` e `dias` e o método `descricao()`. Crie dois objetos e mostre suas descrições.*

Resposta

```
class Pacote:
    def __init__(self, destino, preco, dias):
        self.destino = destino
        self.preco = preco
        self.dias = dias

    def descricao(self):
        return f"{self.destino}: R$ {self.preco:.2f} em {self.dias} dias"

p1 = Pacote("Parintins", 950.00, 3)
p2 = Pacote("Maués", 1350.00, 4)
print(p1.descricao())
print(p2.descricao())
```

Saída esperada:

```
Parintins: R$ 950.00 em 3 dias
Maués: R$ 1350.00 em 4 dias
```

> **Nota ao professor**
> Erros comuns: esquecer o `self` nos métodos ou escrever `nome = nome` em vez de `self.nome = nome` dentro do `__init__`.

Atividade 2

> *Crie a classe `Cliente` com um método `dobrar_dias()` que dobre o atributo `dias`.*

Resposta

```
class Cliente:
    def __init__(self, nome, dias):
        self.nome = nome
        self.dias = dias

    def dobrar_dias(self):
        self.dias = self.dias * 2

cliente = Cliente("Marina", 3)
print(cliente.dias)
cliente.dobrar_dias()
print(cliente.dias)
```

Saída esperada:

```
3
6
```

Desafio do projeto – Classe Usuario

> *Classe `Usuario` (nome, senha) com os métodos `cadastrar()` e `autenticar()`.*

Resposta

```
import os

class Usuario:
    def __init__(self, nome, senha):
        self.nome = nome
        self.senha = senha

    def cadastrar(self, arquivo="usuarios.txt"):
        with open(arquivo, "a", encoding="utf-8") as f:
            f.write(f"{self.nome};{self.senha}\n")

    def autenticar(self, arquivo="usuarios.txt"):
        if not os.path.exists(arquivo):
            return False
        with open(arquivo, "r", encoding="utf-8") as f:
            for linha in f:
                nome, senha = linha.strip().split(";")
                if nome == self.nome and senha == self.senha:
                    return True
        return False

# Teste da classe
Usuario("marina", "1234").cadastrar()
print(Usuario("marina", "1234").autenticar())
print(Usuario("marina", "errada").autenticar())
```

Saída esperada:

```
True
False
```

> **Nota ao professor**
> Lembre os alunos de que guardar senha em texto puro é aceitável **apenas para estudo**.

Aula 19 – Desenvolvimento do cofre digital (oficina de projeto)

Solução de referência do cofre digital

> *Cadastro de usuário com senha, login e mensagem personalizada, tudo persistido em arquivo.*

Resposta

```
import os

class Usuario:
```

```
    def __init__(self, nome, senha):
        self.nome = nome
        self.senha = senha

    def cadastrar(self, arquivo="usuarios.txt"):
        with open(arquivo, "a", encoding="utf-8") as f:
            f.write(f"{self.nome};{self.senha}\n")

    def autenticar(self, arquivo="usuarios.txt"):
        if not os.path.exists(arquivo):
            return False
        with open(arquivo, "r", encoding="utf-8") as f:
            for linha in f:
                nome, senha = linha.strip().split(";")
                if nome == self.nome and senha == self.senha:
                    return True
        return False

def exibir_menu():
    print("\n=== COFRE DIGITAL - AGÊNCIA AMAZÔNIA TUR ===")
    print("1 - Cadastrar usuário")
    print("2 - Entrar")
    print("0 - Sair")
    return input("Escolha: ")

def principal():
    while True:
        opcao = exibir_menu()
        if opcao == "1":
            nome = input("Nome de usuário: ").strip()
            senha = input("Senha: ")
            if nome == "" or senha == "":
                print("Nome e senha são obrigatórios.")
                continue
            if ";" in nome or ";" in senha:
                print("O caractere ; não é permitido.")
                continue
            Usuario(nome, senha).cadastrar()
            print("Usuário cadastrado com sucesso!")
        elif opcao == "2":
            nome = input("Nome de usuário: ").strip()
            senha = input("Senha: ")
            if Usuario(nome, senha).autenticar():
                print(f"Bem-vindo(a) ao cofre digital, {nome}!")
            else:
                print("Usuário ou senha incorretos.")
        elif opcao == "0":
            print("Até logo!")
            break
        else:
            print("Opção inválida.")

principal()
```

Saída esperada – 1ª execução: cadastro, login correto e incorreto, validações e opção inválida:

```
=== COFRE DIGITAL - AGÊNCIA AMAZÔNIA TUR ===
1 - Cadastrar usuário
2 - Entrar
0 - Sair
Escolha: 1
Nome de usuário: marina
Senha: 1234
Usuário cadastrado com sucesso!

=== COFRE DIGITAL - AGÊNCIA AMAZÔNIA TUR ===
1 - Cadastrar usuário
2 - Entrar
0 - Sair
Escolha: 2
Nome de usuário: marina
Senha: 1234
Bem-vindo(a) ao cofre digital, marina!

=== COFRE DIGITAL - AGÊNCIA AMAZÔNIA TUR ===
1 - Cadastrar usuário
2 - Entrar
0 - Sair
Escolha: 2
Nome de usuário: marina
Senha: errada
Usuário ou senha incorretos.

=== COFRE DIGITAL - AGÊNCIA AMAZÔNIA TUR ===
1 - Cadastrar usuário
2 - Entrar
0 - Sair
Escolha: 1
Nome de usuário:
Senha: 123
Nome e senha são obrigatórios.

=== COFRE DIGITAL - AGÊNCIA AMAZÔNIA TUR ===
1 - Cadastrar usuário
2 - Entrar
0 - Sair
Escolha: 1
Nome de usuário: ana;x
Senha: abc
O caractere ; não é permitido.

=== COFRE DIGITAL - AGÊNCIA AMAZÔNIA TUR ===
1 - Cadastrar usuário
2 - Entrar
0 - Sair
Escolha: 9
Opção inválida.

=== COFRE DIGITAL - AGÊNCIA AMAZÔNIA TUR ===
```

```
1 - Cadastrar usuário
2 - Entrar
0 - Sair
Escolha: 0
Até logo!
```

Saída esperada – 2ª execução: o usuário continua cadastrado (persistência):

```
=== COFRE DIGITAL - AGÊNCIA AMAZÔNIA TUR ===
1 - Cadastrar usuário
2 - Entrar
0 - Sair
Escolha: 2
Nome de usuário: marina
Senha: 1234
Bem-vindo(a) ao cofre digital, marina!

=== COFRE DIGITAL - AGÊNCIA AMAZÔNIA TUR ===
1 - Cadastrar usuário
2 - Entrar
0 - Sair
Escolha: 0
Até logo!
```

Nota ao professor

A solução acima é um **ponto de partida**; aceite versões com recursos adicionais (impedir usuário repetido, limitar tentativas, guardar reservas).

Durante a depuração coletiva, os bugs mais comuns são: nomes vazios cadastrados, uso do separador `;` nos dados e `split` quebrando por linha em branco no arquivo.

Aula 20 – Apresentação final, AV4 e encerramento

AV4 – Matriz de correção (projeto e apresentação oral)

Resposta

Avalie o sistema funcionando e a apresentação em dupla. A atribuição de notas é de competência exclusiva do professor.

Critério	Atende plenamente	Atende parcialmente	Não atende
Funcionamento	Cadastro, login e persistência funcionam sem travar.	Funciona com falhas pontuais.	Não executa ou não persiste dados.
Uso dos conteúdos	Usa variáveis, decisões, repetição, estruturas de dados, funções, arquivos e POO.	Deixa de usar parte dos conteúdos.	Uso incorreto ou ausente dos conteúdos.
Organização do código	Nomes claros, comentários, funções e classe bem definidas.	Organização irregular.	Código confuso e sem estrutura.
Apresentação oral	Explica o que fez e por quê, com clareza.	Explica apenas o “o quê”.	Não consegue explicar o próprio código.
Colaboração	Participação equilibrada dos dois integrantes.	Participação desigual.	Trabalho feito por apenas um integrante.

Perguntas orientadoras para a arguição

Resposta

1. Onde está a **classe** do seu sistema e que atributos e métodos ela tem?

2. O que acontece quando o usuário digita uma opção que não existe no menu?

3. Como o programa sabe que o login é válido? Onde os dados ficam guardados?

4. Por que o arquivo é aberto com `"a"` no cadastro e com `"r"` no login?

5. O que vocês melhorariam no sistema com mais tempo? (Pistas de resposta: impedir usuários repetidos, proteger a senha com *hash*, limitar tentativas.)

Resumo das saídas-chave

Conferência rápida dos resultados numéricos mais usados nas correções:

Situação	Resultado esperado
Gasto diário (200 + 50 + 80), 3 dias	R$ 330,00 por dia; total R$ 990,00
Orçamento R$ 1.000 e diária R$ 330	3 dias inteiros; sobram R$ 10,00
Média das notas 8, 7 e 9	8,00
Média dos gastos 990, 1250,5 e 780	R$ 1.006,83
Soma das diárias (100, 120, 140)	R$ 360,00
Código de reserva (Parintins, 3 dias, Marina)	PAR-03-MA
`2 + 3 * 4 ** 2`	50